

Bonus Exercise 3 Report

Algorithms for Uncertainty Quantification (IN2345), SS26

Aleksandra Topalova (03745233), Alexandra Savova (03731070), Julian Kraus (03734087)
Group 36

1 Assignment 1

1.1 Methods

The model is the damped oscillator

$$y''(t) + cy'(t) + ky(t) = f \cos(\omega t), \quad y(0) = y_0, \quad y'(0) = y_1,$$

with $t \in [0, 10]$, $\Delta t = 0.01$, deterministic $\omega = 1$, and quantity of interest $Y = y(10)$. The uncertain inputs are independent uniform random variables:

$$c \sim U(0.08, 0.12), \quad k \sim U(0.03, 0.04), \quad f \sim U(0.08, 0.12), \quad y_0 \sim U(0.45, 0.55), \quad y_1 \sim U(-0.05, 0.05).$$

For the Monte Carlo method from Saltelli et al. (2010), I used the two-matrix design with matrices A , B , and hybrid matrices $A_B^{(i)}$, where column i is taken from B and all other columns from A . The samples were generated as Sobol quasi-random points in dimension $2d$, with $d = 5$, and then split into A and B .

The first-order Sobol' indices were computed with

$$S_i = \frac{\frac{1}{M} \sum_{j=1}^M f(B)_j \left(f(A_B^{(i)})_j - f(A)_j \right)}{\text{Var}(Y)}.$$

For the total-order indices, I used the Jansen estimator recommended in the paper:

$$T_i = \frac{\frac{1}{2M} \sum_{j=1}^M \left(f(A)_j - f(A_B^{(i)})_j \right)^2}{\text{Var}(Y)}.$$

With five uncertain inputs this requires $M(d + 2) = 7M$ ODE solves.

As a comparison, I used a pseudo-spectral polynomial chaos approach with Gaussian quadrature. I computed Sobol' indices from the polynomial chaos expansion using both full tensor-product quadrature and sparse-grid quadrature.

1.2 Results and Discussion

All methods identify the initial velocity y_1 as the dominant input, contributing about 87% of the variance in $y(10)$. The spring coefficient k is the second most important input. The damping coefficient c , forcing amplitude f , and initial position y_0 have much smaller effects.

The full-grid and sparse-grid pseudo-spectral results are almost identical, which indicates that the oscillator response is smooth enough for a low-order polynomial chaos approximation. The Saltelli quasi-Monte Carlo estimates have the same ranking and very similar magnitudes, but they are slightly noisier because they converge statistically with the number of samples. The pseudo-spectral method is much cheaper for this problem: for $K = N = 4$, the Saltelli method needs 7168 ODE solves, while the full-grid PCE method needs 1024 and the sparse-grid PCE method only 259.

Table 1: First-order and total-order Sobol' indices for $K = N = 4$.

Method	Input	S_i	T_i	ODE solves
Saltelli QMC/Jansen	c	0.0200	0.0216	7168
Saltelli QMC/Jansen	k	0.1017	0.1067	7168
Saltelli QMC/Jansen	f	0.0030	0.0030	7168
Saltelli QMC/Jansen	y_0	0.0000	0.0004	7168
Saltelli QMC/Jansen	y_1	0.8731	0.8763	7168
PCE full grid	c	0.0196	0.0215	1024
PCE full grid	k	0.1026	0.1064	1024
PCE full grid	f	0.0030	0.0030	1024
PCE full grid	y_0	0.0001	0.0004	1024
PCE full grid	y_1	0.8691	0.8744	1024
PCE sparse grid	c	0.0196	0.0215	259
PCE sparse grid	k	0.1026	0.1064	259
PCE sparse grid	f	0.0030	0.0030	259
PCE sparse grid	y_0	0.0001	0.0004	259
PCE sparse grid	y_1	0.8691	0.8744	259

Table 2: Computational cost in model evaluations.

Method	Configuration	Samples/nodes	ODE solves
Saltelli QMC/Jansen	$M = 243$	243	$243(5 + 2) = 1701$
Saltelli QMC/Jansen	$M = 1024$	1024	$1024(5 + 2) = 7168$
PCE full grid	$K = N = 3$	$3^5 = 243$	243
PCE sparse grid	$K = N = 3$	63	63
PCE full grid	$K = N = 4$	$4^5 = 1024$	1024
PCE sparse grid	$K = N = 4$	259	259

2 Assignment 2

I discretized the domain $[0, 1]^2$ with a 40×40 midpoint grid. For grid points x_i , the covariance matrix was assembled as

$$\hat{C}_{ij} = C(x_i, x_j).$$

The mean function was set to $m(x) = 0.1$, and the length scale was set to $l = 0.2$. Samples were generated by Cholesky factorization,

$$\hat{C} + \epsilon I = LL^T,$$

where $\epsilon = 0$ unless a small diagonal jitter is needed for numerical stability. For $\Psi \sim N(0, I)$, the random field samples are then given by

$$G = \hat{m} + L\Psi.$$

Figures 1 and 2 show that the two covariance functions produce visibly different regularity. In Figure 1, the exponential covariance samples are rougher and show more local variation. In Figure 2, the squared-exponential covariance samples are much smoother and form larger coherent regions. This is expected because the squared-exponential kernel decays smoothly with squared distance and therefore imposes stronger spatial smoothness. The exponential kernel allows less regular sample paths and therefore produces more local fluctuations.

This difference is useful in modelling. The exponential covariance is suitable when the modelled quantity may vary irregularly or have rough local fluctuations, while the squared-exponential covariance is more suitable when the quantity is expected to vary smoothly over space.

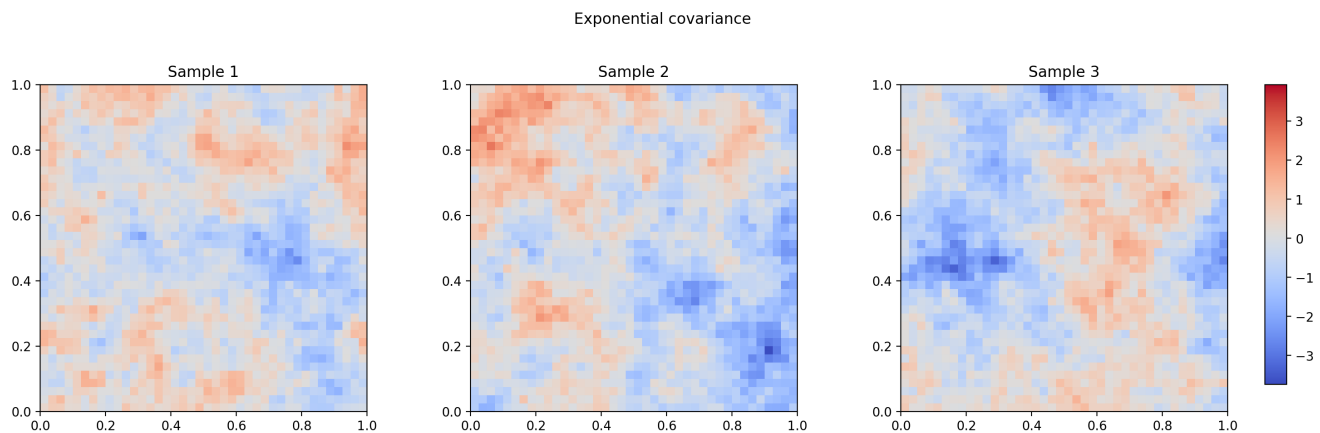


Figure 1: Three Gaussian random field samples with exponential covariance $C_1(x, y) = \exp(-\|x - y\|_2/l)$.

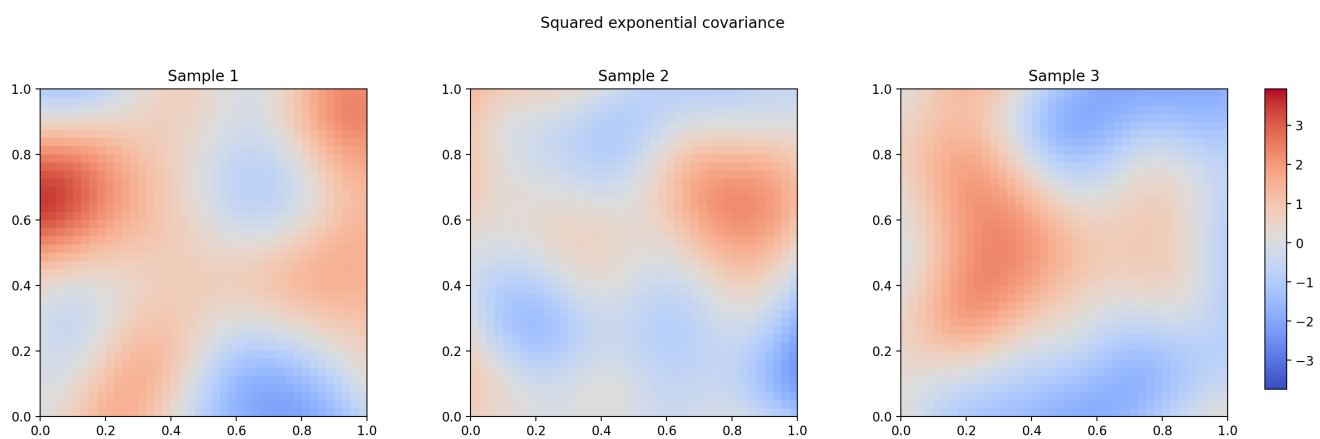


Figure 2: Three Gaussian random field samples with squared-exponential covariance $C_2(x, y) = \exp(-\|x - y\|_2^2/(2l^2))$.

3 Assignment 3

3.1 Assignment 3.1

This subsection first discusses the implementation of the two Wiener process simulation methods (direct definition realization and Karhunen–Loève expansion) and then elaborates on what was observed when comparing their results.

3.1.1 Implementation

The implementation for the process value generation resides entirely in the `WienerProcess` class. Its helper functions allow us to generate the direct definition realization, an approximation using the KL expansion, the eigenvalues and eigenfunctions.

- Direct definition realization - For each sample we project a trajectory in time steps. At each step we increment with a random value and form the next step as a cumulative sum of the previous steps. In the end, we shift by the mean.
- Approximation using the KL expansion - Follows directly the approximation formula below for each time step (total of N) for each sample, but with matrix multiplication for speedup. Here we changed the term from $(m+0.5)$ to $(m-0.5)$ since our indexing starts at 1. We again shift with the mean after computing with the formula. In the formula we scale with T (useful for $T > 1$) appropriately to reuse in Assignment 3.2.

$$W(t) \approx \mu + \sum_{m=1}^M \frac{\sqrt{2T}}{(m - \frac{1}{2})\pi} \sin\left(\frac{(m - \frac{1}{2})\pi t}{T}\right) \zeta_m, \quad \zeta_m \sim \mathcal{N}(0, 1).$$

- KL eigenvalues - Follows the formula below exactly with indexing 1 to M

$$\lambda_m = \frac{1}{(m - 0.5)^2 \pi^2}, \quad (1)$$

- KL eigenfunctions - Similarly to the eigenvalues, follows the formula below exactly with indexing 1 to M . Instead of taking a singular point t , we take a `t_grid` and return a callable function.

$$\phi_m(t) = \sqrt{2} \sin\left(\pi t \left(m - \frac{1}{2}\right)\right) \quad (2)$$

Configuration We simulate with the following configuration of parameters (as per the worksheet):

- `T = 1` - Because $T \in [0, 1]$ in the worksheet and T serves as an upper time period bound in the code
- `n_points = 1000` - Since $N = 1000$ in the worksheet
- `t_grid = np.linspace(0, T, n_points)` - In template
- `Ms = [10, 100, 1000]` - Per worksheet, we need to test with these 3 values for M
- `seed = 42` - Default chosen seed
- `n_samples = 1` - Because the worksheet asks for a single realization of the Wiener process

Additionally, the random number generator was reinitialized for computing the KL approximation to ensure they all use the same seed and values. Here the difference between N and M becomes clearer: N is the amount of time points/steps that we make for 1 realization and M is a KL-specific parameter that represents the number of basis functions or expansion terms in the series when approximating (relatively analogous to a Taylor series).

3.1.2 Observations

In Figure 3, we can see that the direct definition implementation of the Wiener process creates a jagged continuous line. The general graph tendency exists because of the cumulative nature of the definition. However, every single time step gets its own independent random draw, which causes the jaggedness.

What we also notice is that despite using the same seed for the random number generation, the KL approximations don't look anything like the direct definition. This difference is because they sample completely random variables, i.e. in KL we sample ζ , which is only a multiplier, but not the entire step increment.

When comparing the KL approximations with $M = [10, 100, 1000]$, we see that a bigger M converges in a more accurate representation of the direct definition behavior. Since the actual value of W_t is an infinite sum, which we truncate to M terms for the approximation, more KL terms give us a better accuracy as they add finer-grained details.

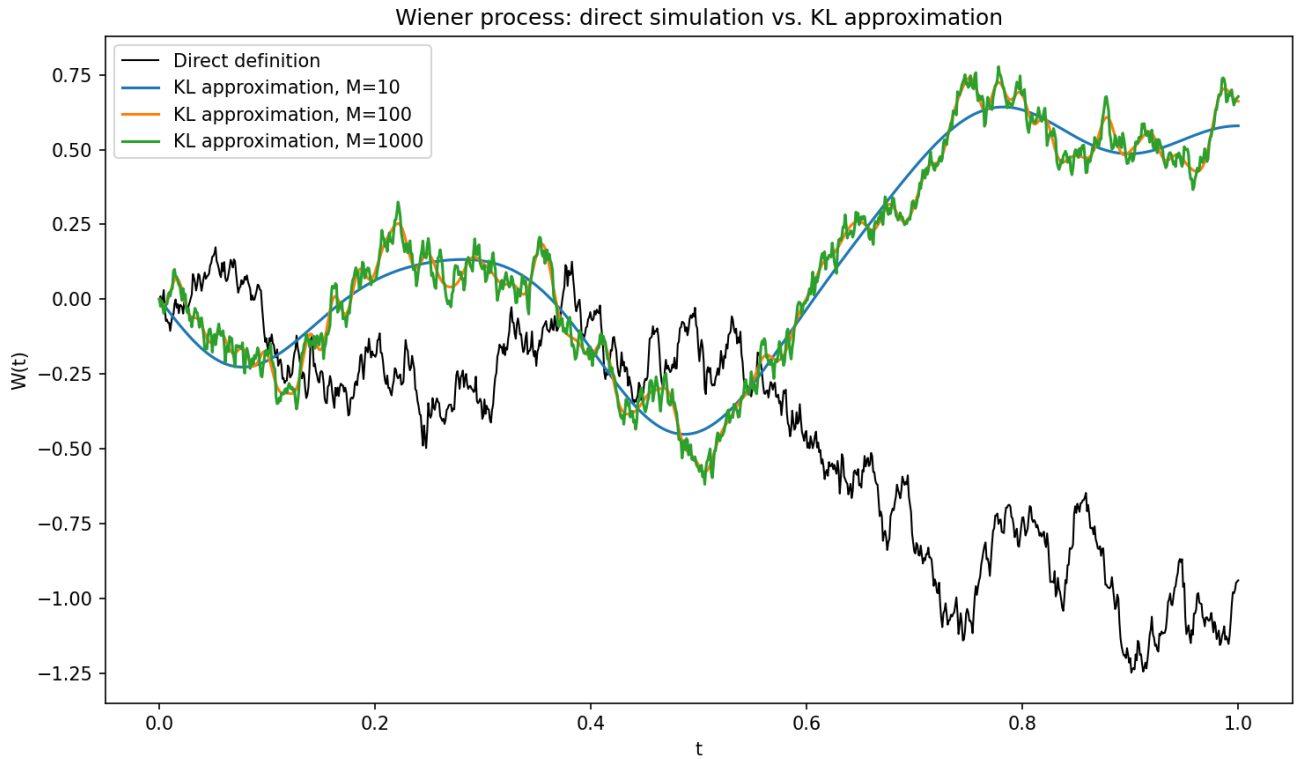


Figure 3: Comparison between the Wiener direct definition simulation and the KL approximations with $M = [10, 100, 1000]$

Looking at Figures 4 and 5, we can see that each eigenvalue decrease polynomially and converge to 0, i.e. the first eigenvalue is the biggest and the last one is the smallest. This behavior is a direct consequence of the Formula 1, where m is in the denominator and a bigger m would yield a smaller λ_m .

On Figure 5, we can also see more clearly that a bigger m gives us tighter periodicity, which is because in Formula 2 m ($m \geq 1$) acts as a multiplier in the $\sin(\cdot)$ function that increases the frequency/shortens the period.

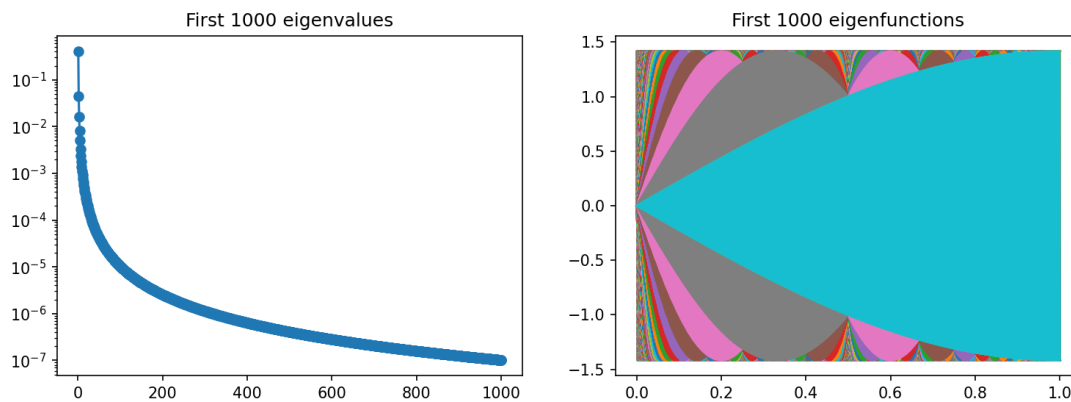


Figure 4: KL eigenvalues and eigenfunctions for $M = 1000$

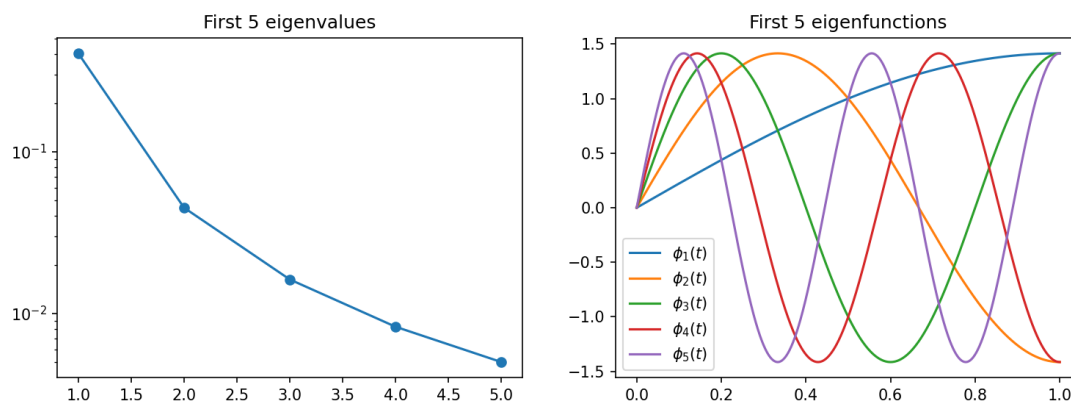


Figure 5: KL eigenvalues and eigenfunctions for $M = 5$

3.2 Assignment 3.2

In the following, we will discuss our findings and present the implementation of assignment 3.2.

3.2.1 Results

Table 3: Mean and Variance of solutions at $T=10$ for different Wiener process implementations

Method	Mean	Variance
Standard WP	-0.42748538754000037	6.803879571502152
KL, $M=5$	-0.32791226576256555	7.033168222664913
KL, $M=10$	-0.4165152554396343	6.711735249937001
KL, $M=100$	-0.30603320430438763	6.758075228838829

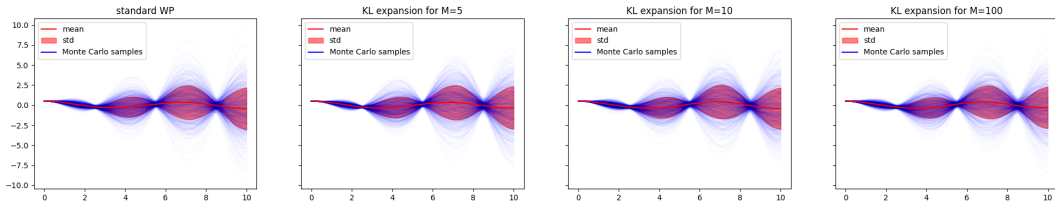


Figure 6: Comparing different Wiener process implementations. Monte Carlo samples and quantities of interest: mean and standard deviation

In Table 3, we show the quantities of interest, mean and the variance, for $y(10)$ with f as the direct implementation of the Wiener process and as different Karhunen-Loève discretisations with $M = [5, 10, 100]$ respectively. We observe that the mean values vary across the Karhunen-Loève discretisations. Nonetheless, the differences between those values and the mean of the standard Wiener process are small and could be attributed to noise and rounding errors. Therefore, in the context of the linear damped oscillator problem, the KL discretisations with even a small number of terms are a good alternative to the direct implementation, where the mean is the quantity of interest and a small error is acceptable. The variance for the KL discretisations converges towards the direct implementation variance with higher M . Therefore, $M = 100$ is the better choice for KL in case the variance is the quantity of interest and only a small error is acceptable.

In Figure 6, we visually compare the results for the direct Wiener process implementation and the three Karhunen-Loève discretisations. We see that the four graphics look identical. Hence, even a small number of KL terms, such as $M = 5$, is sufficient to visualise the trends in the Wiener process implementation of f for the linear damped oscillator problem.

Nonetheless, the computations were time-consuming (on average, about 2.5 minutes wall-clock time per experiment run, although we did not time it precisely). Therefore, for more complex problems, other methods to compute the mean and variance of the solution should be considered rather than performing the 1000 oscillator evaluations for Monte-Carlo.

Methods other than Monte Carlo that could be employed to compute the mean and variance of the solution, and how we would employ them:

- Due to the linearity properties of our model and since the covariance function $C(t, s) = \min(t, s)$ of a Wiener process is analytically solvable, we could rewrite the oscillator ODE, substitute the Wiener process representation as f and mathematically transform the mean and variance computations into deterministic formulas. Thus, we would remove the necessity for Monte Carlo sampling or any sampling.
- Going back to past ideas: To reduce the number of samples for which the oscillator is evaluated, we could try Quasi Monte-Carlo sampling (e.g. with Halton sequence) instead of Standard Monte-Carlo sampling for the mean and variance computations. This suggestion is similar to the topics we explored in bonus exercise 1, but we need to check whether it would be as effective for the Wiener process as it was for a random variable.

- Alternatively, we could evaluate the oscillator at Gaussian quadrature nodes, fit an interpolant through those node evaluations and derive mean and variance from the interpolant. This is similar to bonus exercise 2.

3.2.2 Implementation

To begin with, the implementation of assignment 3.2 also relies on the helpers of the `WienerProcess` class, specifically the `generate` and `approximate_kl` methods. The code logic is:

- Definitions of the parameters for the oscillator and the experiment, as listed in the assignment sheet and similar to previous bonus sheets. For reproducibility, we reuse the random generator **seed 42** that we used in our previous bonus exercise solutions.
- Where needed, definitions of dictionaries as data structures to save both the results and the context of the different experiment runs.
- Generation of Wiener process samples for the callable function f : $N = 1000$ samples generated using the standard definition, and $N = 1000$ samples generated using KL expansion with $M = [5, 10, 100]$ terms. By definition of the oscillator model, f is a function. Thus, first, we generate samples for the values of f over the time grid. Then, we transform those samples into callable functions that can be used to evaluate the Wiener Process at a given time t through interpolation.
- Simulating the oscillator with the generated samples and in parallel calculation of the mean and standard deviation of the results for $y(T = 10)$. To clarify, the assignment sheet requirement is to present results about the mean and variance, as in Table 3, but the code template for `compute_metrics` and the compatibility with the provided plotting helper `plot_solutions` require a computation of standard deviation rather than variance. Hence, we first calculate the standard deviation with `ddof=1` (for the sample rather than the population estimate) and later compute the variance as the square of the standard deviation.
- Finally, plots of the experiment results, including mean and standard deviation, with the provided `plot_solutions` helper. Printing out mean and variance (square of the standard deviation) information for $y(T = 10)$.